



华中科技大学计算机科学与技术学院 2021~2022 第二学期

“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试日期 2022-05-21 考试时长 150 分钟

专业班级 学 号 姓 名

题号	一	二	三	四		总分	核对人
分值	40	20	20	20		100	
得分							

分 数	
评卷人	

一、程序运行的基本原理问答 (共 40 分)

解答内容不得超过装订线

对如下 C 语言程序, 用 VS2019 (Intel CPU) 编译、链接、调试运行。

```
#include <stdio.h>
int fmax(int x, int y)
{
    int u;
    if (x > y) u = x;
    else u = y;
    return u;
}
int main(int argc, char* argv[])
{
    int a = 100; // 0x64
    int b = 0x12345678;
    int r = 0;
    char msg[6] = "abcde"; // 'a'的ASCII是 0x61
    r = fmax(a, b);
    msg[17] = 'l'; // 'l'的ASCII是 0x31
    r = *(int*)(msg+1);
    return 0;
}
```

在调试时, 在监视窗口中看到 变量 a 的地址 (即 &a) 为 0x010ffe98; 变量 b 的地址 (即 &b) 为 0x010ffe94; 变量 r 的地址为 0x010ffe90, 数组 msg 的起始地址为 0x010ffe88。

1、填写执行到 “r=fmax(a,b);” 时, 内存中数据存放的结果。要求以字节为单位、用 16 进制数的形式填空, 最左边是内存窗口显示的内存地址。(10 分)

0x010ffe88 61 62 63 64 65 00 XX XX
 0x010ffe90 00 00 00 00 78 56 34 12
 0x010ffe98 64 00 00 00 XX XX XX XX

2、数据传送指令解读（10 分，每空 2 分）

语句 “int a = 100;” 的反汇编指令为： `mov dword ptr [ebp-4], 64h`

根据观察到的 a 的地址，推断执行该语句时，(ebp)= 0x 10ffe9c，该指令的机器码为 C7 45 FC 64 00 00 00，其中 FC 对应的是指令中 -4 的编码。

根据观察到的 a 与 b 的地址关系，推断出 “int b = 0x12345678;” 对应的反汇编指令为：

mov dword ptr [ebp-8], 12345678h

执行 “msg[17]='1;” 后，变量 a 中的值为 0x 3164。

执行 “r = *(int *) (msg+1);” 后，r 中的值为 0x 65646362。

3、函数调用语句解读（10 分，每空 2 分）

对于语句 “r=fmax(a,b);” 对应的反汇编代码（最左边的是机器指令的地址）如下。

```
0072162B  mov     eax, dword ptr [ebp-8]
0072162E  push    eax
0072162F  mov     ecx, dword ptr [ebp-4]
00721632  push    ecx
00721633  call    007212B2
00721638  add     esp, 8
0072163B  mov     dword ptr [ebp-0Ch], eax
```

	0x010ffe28
	0x010ffe2c
0x00721632	0x010ffe30
0x00000064	0x010ffe34
0x12345678	0x010ffe38
XXXXXXXXXX	0x010ffe3c

设执行 “r=fmax(a,b);” 之前，(esp) = 0x010ffe3c

① 在表格的适当位置填写 刚进入函数 fmax 内部时，堆栈中存放的相关数据。

② 刚进入函数 fmax 内部时，(esp)= 0x 010ff330

③ 执行 “add esp, 8” 之后，(esp) = 0x 0x10ffe3c。

4. 函数体的指令解读 (10 分，每小题 2 分)

函数体对应的反汇编代码有：

```
        push    ebp
        mov     ebp, esp      .....
    if (x > y) u= x;
007215C3  mov     eax, dword ptr [ebp+8]
007215C6  cmp     eax, dword ptr [ebp+0Ch]
007215C9  jle     007215D3
007215CB  mov     eax, dword ptr [ebp+8]
007215CE  mov     dword ptr [ebp-4], eax
007215D1  jmp     007215D9
        else u = y;
007215D3  mov     eax, dword ptr [ebp+0Ch]
007215D6  mov     dword ptr [ebp-4], eax
        return u;
007215D9  mov     eax, dword ptr [ebp-4] .....
        pop     ebp
        ret
```

- ① 执行到 “cmp eax, dword ptr [ebp+0Ch]” 指令时, (ebp)=0x_010ff32c。
- ② 函数参数 x 的地址 (即&x) 是 0x_010ff334。
- ③ 执行 “cmp eax, dword ptr [ebp+0Ch] ” 后, CF=_1_, SF=_1_, ZF=_0_, OF=_0_。
- ④ 执行 RET 指令时, CPU 所做的工作是_将栈顶双字出栈到 EIP_____。
- ⑤ 函数返回结果的方法是_使用 EAX 传递返回整数值_____。

分 数	
评卷人	

二、程序阅读与理解 (共 20 分)

1、阅读下面的程序, 回答问题 (10 分)。

```
.686P
.model      flat, c
includelib kernel32.lib
includelib libcmnt.lib
includelib legacy_stdio_definitions.lib
printf      proto :ptr sbyte, :vararg
ExitProcess proto stdcall :dword

.data
    nums    sdword 1234, -1, 100, -20, -2
    len     = ($-nums)/4
    fmt     db "%d ", 0dh, 0ah, 0

.code
main proc c
    mov     edx, 0
    mov     ecx, len          ; ①
    lea     esi, nums         ; ②
process:
    cmp     dword ptr [esi], 0
    jge     next              ; ③
    inc     edx
next:
    add     esi, 4
    dec     ecx
    jne     process
    invoke  printf, offset fmt, edx
    invoke  ExitProcess, 0
main endp
end
```

1) 上述程序的功能是什么? 运行后, 屏幕上显示的是什么? (2 分)

统计 nums 中, 负数的个数, 保存到 edx 中, 并显示 edx 的值。显示 3

2) 若将 ③ 处的语句改为 “jae next”, 程序运行的结果是什么? (2 分)

显示 0

3) 若标号 process 写到 ②处语句前, 程序运行的结果是什么? 为什么? (3 分)

显示 0, 始终比较第一个元素

4) 若标号 ①处语句写到标号 process 之后, 程序运行会出现什么现象? 为什么? (3 分)

```
process: mov ecx, len
        cmp dword ptr [esi], 0 .....
```

死循环。每次循环未修改 ecx 的值。

2. 程序填空 (10 分, 每空 1 分)

在一个以 0 结束的字符串中, 将所有的大写字母转换为对应的小写字母, 并将转换结果、以及修改的字母个数输出。

```
.....
.data
    buf db "abcDEFgh", _0____
    fmt db "%s %d", 0

.code
main proc c
    mov esi, _offset buf_ ; esi 存放待访问字符的地址
    mov ecx, _0 ; ecx 存放修改字母的个数
next:
    mov al, [esi]
    cmp al, 0
    je exit_____
    cmp al, 'A'
    _jb cont_____
    cmp al, 'Z'
    _ja cont_____
    add al, __'a' - 'A'
    mov _[esi]____, al
    inc ecx_____
cont:
    inc esi
    jmp next_____
exit:
    invoke printf, offset fmt, offset buf, ecx
    invoke ExitProcess, 0
main endp
end
```

分 数	
评卷人	

三、程序优化（共 20 分）。

- 1、对 C 程序进行编译生成 Release 版程序时，编译器会做一些优化工作。请举出 5 个优化场景，简要说明做了什么优化，以及为什么能加快执行速度。（10 分）

使用寄存器作为循环计数器；

使用寄存器作为数组下标，进行变址寻址；

使用寄存器传递参数；

使用带比例因子的变址寻址操作数，获取 EA，代替算术运算；

使用宽字节类型的寄存器，进行数组元素的操作，减少循环次数；

使用顺序访问数组元素的方式，代替循环访问数组元素；

- 2、如下的 C 语言程序段（32 位段）实现了“统计一个字符串 buf 中小写字母的个数，并将其放入 count 中”的功能。其编译后调试版本的汇编语言代码如下(注：斜体部分为 C 语句；在反汇编时，勾选显示符号名)。（10 分）

```

int count = 0;
00A616A0  mov     dword ptr [count],0
int i;
for (i = 0; buf[i] != 0;i++)
00A616A7  mov     dword ptr [i],0
00A616AE  jmp     fcount+59h (0A616B9h)
00A616B0  mov     eax,dword ptr [i]
00A616B3  add     eax,1
00A616B6  mov     dword ptr [i],eax
00A616B9  mov     eax,dword ptr [i]
00A616BC  movsx   ecx,byte ptr buf[eax]
00A616C1  test    ecx,ecx
00A616C3  je      fcount+8Ah (0A616EAh)
    if (buf[i] >= 'a' && buf[i] <= 'z')
00A616C5  mov     eax,dword ptr [i]
00A616C8  movsx   ecx,byte ptr buf[eax]

```

```

00A616CD  cmp     ecx,61h
00A616D0  jl      fcount+88h (0A616E8h)
00A616D2  mov     eax,dword ptr [i]
00A616D5  movsx   ecx,byte ptr buf[ecx]
00A616DA  cmp     ecx,7Ah
00A616DD  jg      fcount+88h (0A616E8h)
        count++;
00A616DF  mov     eax,dword ptr [count]
00A616E2  add     eax,1
00A616E5  mov     dword ptr [count],eax
00A616E8  jmp     fcount+50h (0A616B0h)

```

(1) 指出该段程序执行效率不高的原因 (2 分)。

使用局部变量作为循环计数器；使用局部变量作为数组下标，进行变址寻址；
在条件表达式中，对同一个变量 `buf[i]`，进行了两次赋值

(2) “00A616D0 `jl fcount+88h (0A616E8h)`” 处指令的机器码为 7CH 16H，解释 16H 代表的含义 (2 分)

为跳转目的地址到该指令的距离： $0A616E8h - (0A616D0h + 2)$

(3) 改编相应的汇编语言程序，以提高程序的执行效率。(6 分)

要求写出采取的优化方法，在优化程序段中，可以用标号来代替转移指令的目的地址。

```

count: edx
i: ecx
buf[i]: al

mov edx, 0
mov ecx, 0
next: mov al, buf[ecx]
      cmp al, 0
      je exit
      cmp al, 'a'
      jb cont
      cmp al, 'z'
      ja cont
      inc edx
cont: jmp next
exit: mov count, edx

```

分 数	
评卷人	

四、设计题（20 分）

在 BUF1 为首址的字类型存储区中，存放了 M 个互不相同的数据；在 BUF2 为首址的字类型存储区中存放了 N 个数据。用汇编语言编写完整的程序，统计在两个存储区中同时出现的数据个数，存放到字类型变量 COUNT 中，最后在屏幕上输出 COUNT 的值。

要求：

- (1) 简要描述设计思想，给出寄存器分配方案。
- (2) 用子程序 FIND 判断某个字类型数据是否在某存储区中出现；描述其入口参数、出口参数。
- (3) 画出程序的流程图。
- (4) 程序完整（包括堆栈段、数据段、代码段定义等），至少给出 4 条必要的注释。
- (5) BUF1、BUF2 中的数据在定义时直接给出初值，无需输入；M、N 自己给定；子程序 FIND 中不允许使用全局变量。

(1)

esi: BUF1 寻址、BUF2 寻址

ecx: 外循环、内循环

(2) FIND 入口参数：

ax: 待查找字

ebx: 存储区首址

edx: 存储区长度

出口参数：

ax: 1 表示找到；0 表示未找到

